Unit-3

PROGRAMMING TECHNIQUES

Compiled by Er BRJ

Er. Bhisma Raj Joshi
Department of Computer Engineering
[NAST]

@Er_BHISMARAJ

OBJECTIVES

- INTRODUCTION TO PROGRAMMING TECHNIQUES
- TYPES, MODULAR, TOP-DOWN, BOTTOM-UP, STRUCTURE AND OBJECT-ORIENTED APPROACH
- ADVANTAGES AND DISADVANTAGES
- PROGRAM DEVELOPMENT METHODS(WATERFALL MODEL, PROTOTYPE MODEL, SPIRAL MODEL, ITERATIVE MODEL, V-MODEL, RAD MODEL BIG BANG MODEL, AGILE MODEL)
- COHESION AND COUPLING AND THEIR TYPES,USES
- LOGICS OF PROGRAM DEVELOPMENT (SEQUENTIAL, SELECTION, ITERATION AND RECURSION)
- COMMUNICATION BETWEEN MODULES

Compiled by Er BRJ

@Er_BHISMARAJ

EXAM POINT OF VIEW

8HR

2 Long Questions 7+8 =15

1 Short Note 1*5 = 5

Compiled by Er BRJ

@Er_BHISMARAJ

PROGRAMMING TECHNIQUES

- As we know program is the series of instructions and procedures which are written to perform specific tasks.
- Simply programmer can use any logics and methods to develop the program so programming techniques are the logics, procedures and methodologies of the programming which are used by the programmer to design and develop the program easily.
- These techniques help developers create reliable, maintainable and scalable software product.

Compiled By Er BRJ

@ER_BHISMARAJ

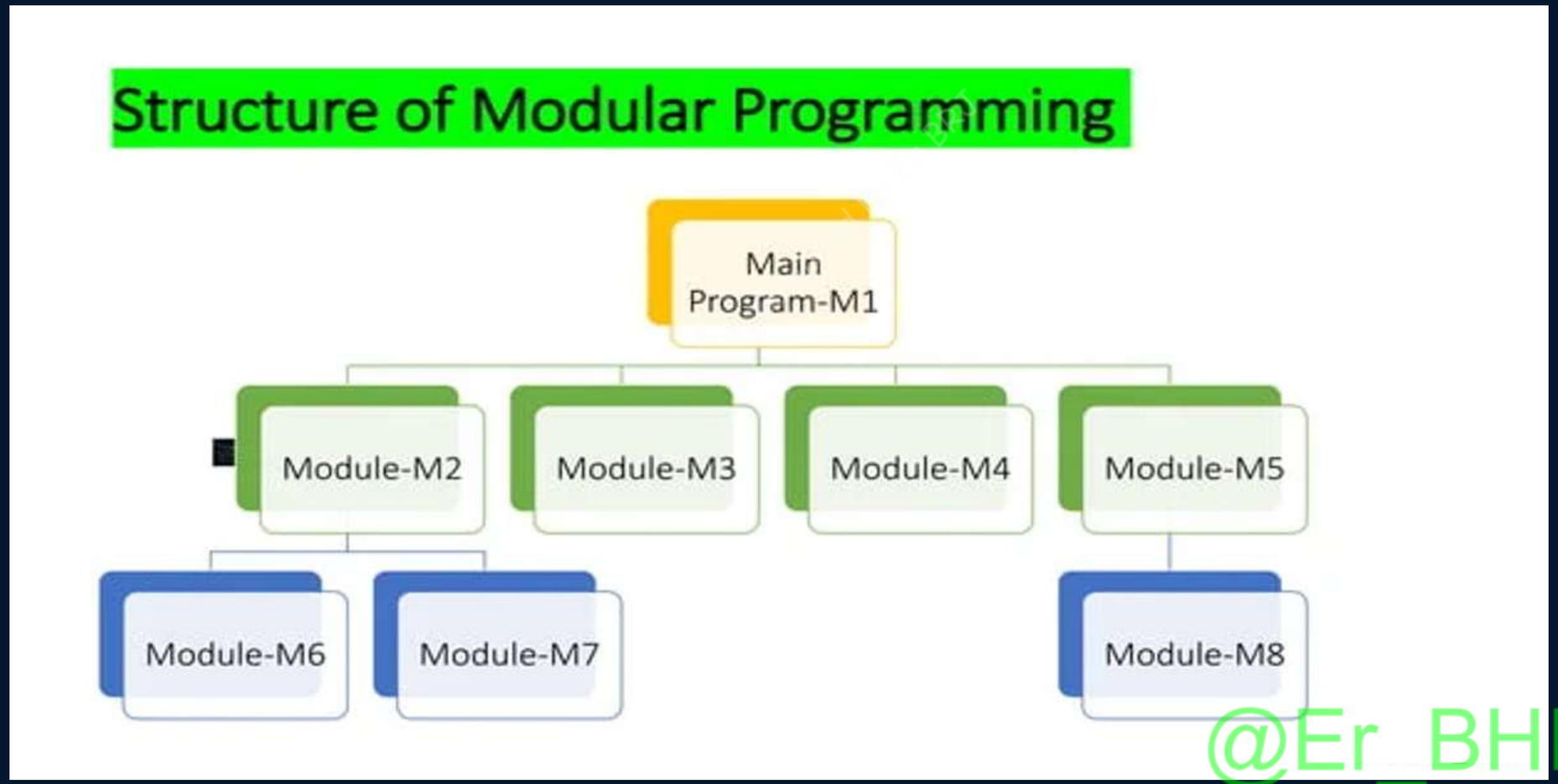
TYPES / APPROACHES OF PROGRAMMING TECHNIQUES

1. Modular Approach

- When a program becomes very larger and complex, it becomes a very difficult task for the programmer or developer to design, test and debug such a program.
- The modular approach of programming involves breaking down a software program into separate, smaller and manageable components which are simply called modules.
- Every module handles a specific part of the program's functionality and can be developed, tested and maintained independently. For example, we can consider an e-commerce website.

Compiled by ER_BHISMARAJ
@Er_BHISMARAJ

- This is an important technique for programming which is mostly used for designing the large and complex programs. This technique is also can be used for shorter programs of microprocessor. The division technique is used in modular programming.



@Er_BHISMARAJ

- Here, a large program divided into different modules and sub modules. Each module encompasses a specific function within the program.
- This technique allows several programmers to code simultaneously and decrease the debugging time. Function of each module can be logically separated.

Compiled by Er BRJ

ASSIGNMENT 1

WRITE DOWN THE MAJOR ADVANTAGES AND DISADVANTAGES OF MODULAR APPROACH.

@Er_BHISMARAJ

2. Top-Down Approach

- The top-down programming technique is a problem-solving approach that starts with a high-level concept of the problem and breaks it down into smaller, more manageable parts. This method is also known as **divide and conquer**.

- In this model, an overview of the system is formulated without going into details for any part of it. After overview formulation, each parts are defined in more details until the entire specification is detailed enough to validate the model.

Compiled By Er BRJ

@Er_BHISMARAJ

- It is generally used by structured programming languages such as C, COBOL, FORTRAN, etc.

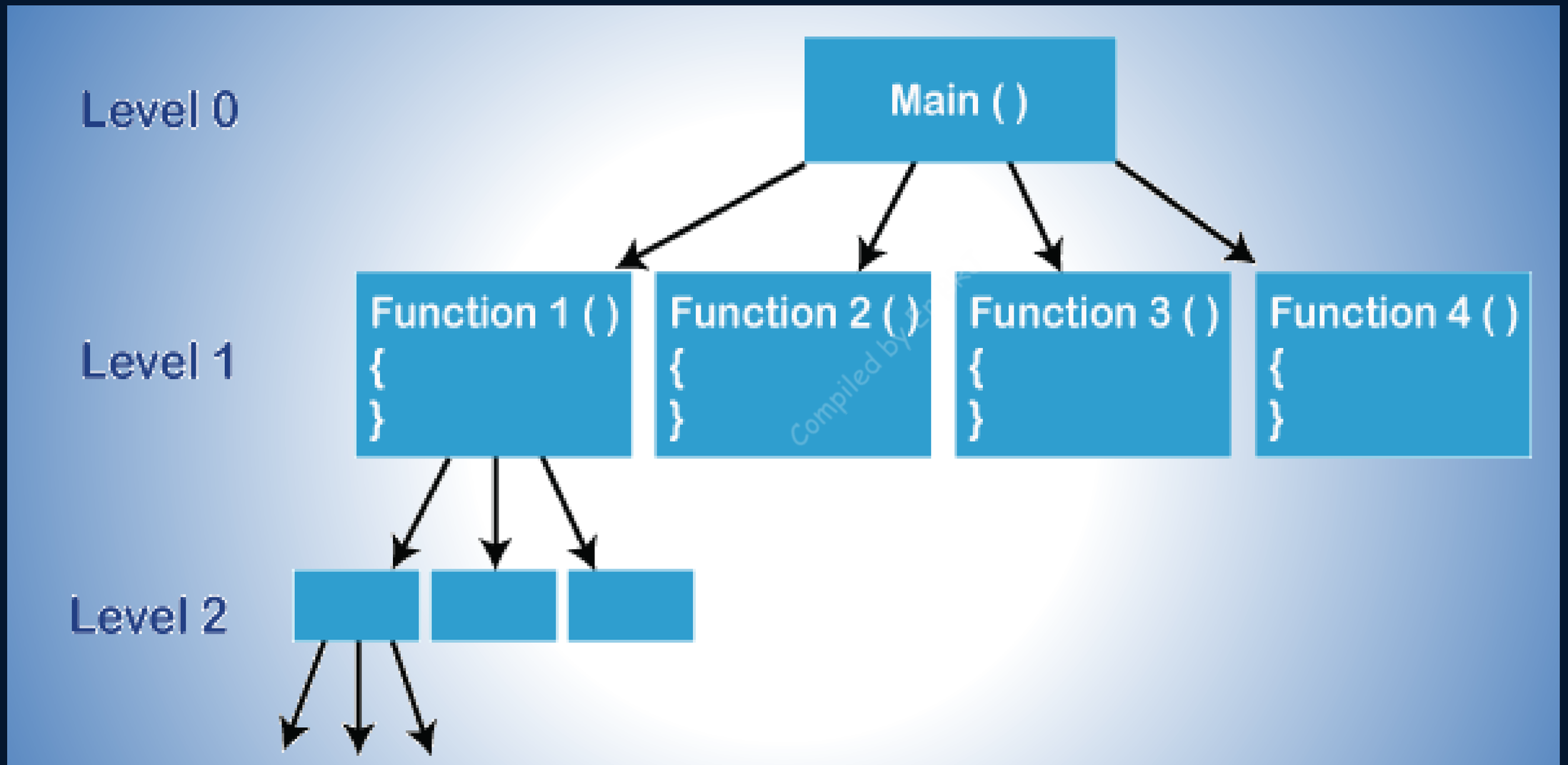


Fig:- Top-Down approach of programming

@Er_BHISMARAJ

Advantages

- It provides a clear roadmap of the project from start to finish.
- Makes it easier to understand the system’s structure.
- Facilitate parallel development by different team members.
- Modules can be tested independently before integration.
- Bugs are easier to isolate and fix.

Compiled by Er BRJ

Disadvantages

- Needs a clear vision or understanding of the problem before starting.
- Data are not hidden, so can be processed by any function.
- Significant time is spent on planning and designing before coding begins.

@Er_BHISMARAJ

3. Bottom-Up Approach

- The bottom-up approach in programming and software development is a methodology that starts with designing and implementing the most basic or low-level components first. These components are then integrated to form more complex systems and, eventually, the entire application.
- This approach is the opposite of the top-down approach and is particularly useful in situations where specific lower-level functionalities are well-understood.
- In this technique the individual base elements of the program are first specified and then linked together with each other and with the main program.

Compiled by Er BRS

@Er_BHISMARAJ

- When designing mechanical systems, an Engineer uses a bottom-up approach. This means they create individual parts first and then put them together to form the final system, without worrying about how these parts will work together.

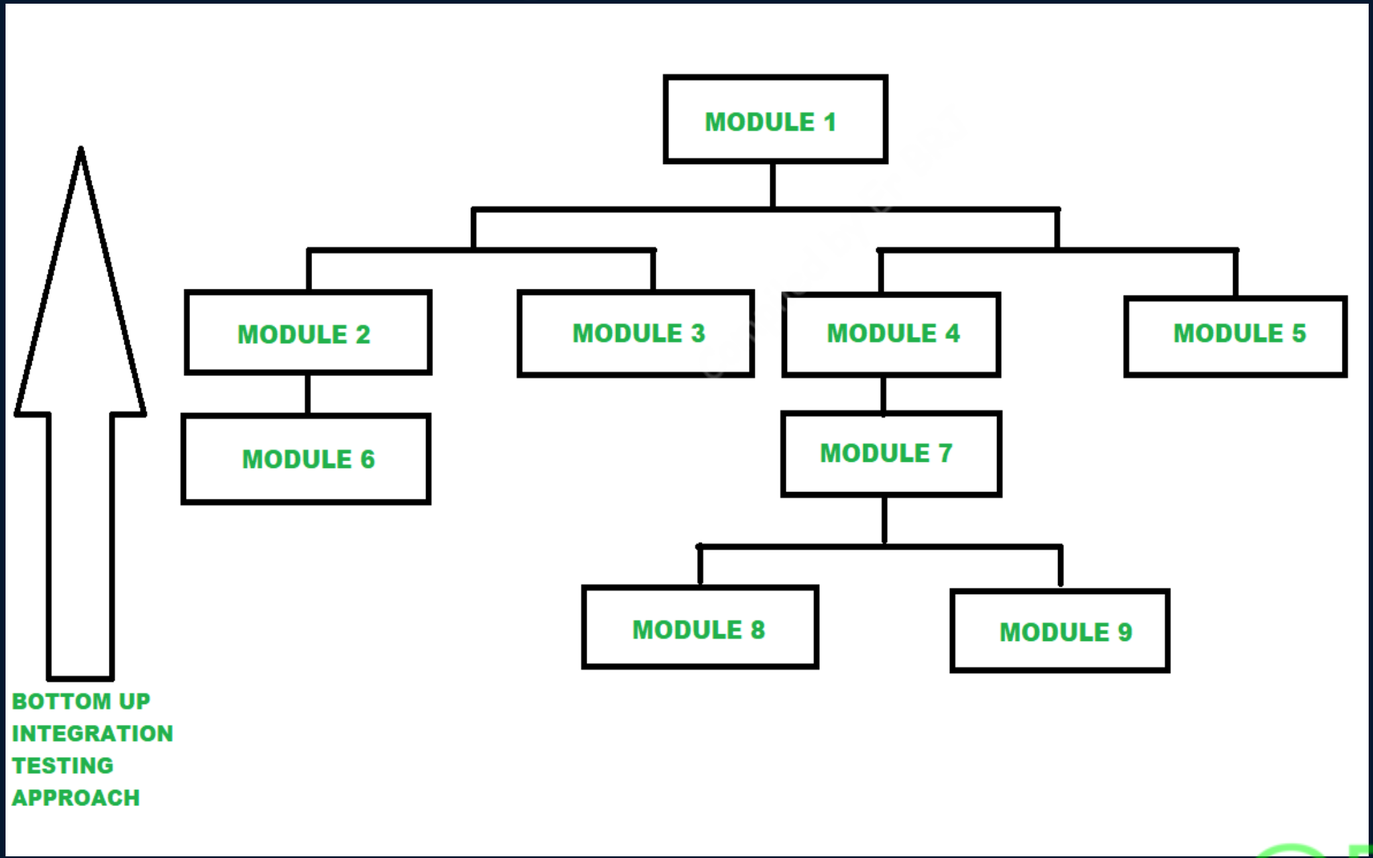


Fig:- Bottom-Up approach of programming

@Er_BHISMARAJ

Advantages

- creating reusable modules or functions that can be used in different contexts.
- Small components can be developed and tested in isolation.
- Changes or improvements can often be made to individual components without affecting the rest of the system.
- Each component is tested as it is built, reducing the risk of errors in final system.

Compiled by Er BRJ

Dis-advantages

- Combining many small components, can be complex and time-consuming.
- Improper management of dependencies between components can create issues during integration.
- High quality bottom-up approach solutions are hard to construct.
- The progress is cannot be verified easily at the mid stage.

@Er_BHISMARAJ

Assignment 2

Write down the major differences between top-down and bottom – up approach.

Compiled by Er BR

@Er_BHISMARAJ

4. Structured Programming Approach

- The long and complex program may be well understood by the programmer who developed but not understood by the person who had maintained or used the program. To overcome this difficulty a technique developed to write a program which called Structured Programming Technique.

- This is the method of programming where the programmer divides his source code of program in to logically structured chunks of code. Structuring a program helps to break down a program into understandable chunks.

Compiled by Er_BJ

@Er_BHISMARAJ

- The structured program mainly consists of three types of elements:

a) Sequence Statements:- In which statements are executed one after another in sequence manner.

b) Selection Statements:- The selection control structures allow the program to make decisions and execute different code blocks based on certain conditions. It includes if, if else, else if ladder and nested if else statements.

Compiled by Er. BRJ

c) Iteration(looping) Statements:- An **iterative or looping structure** is a control structure in programming that repeatedly executes a block of code as long as a specified condition is true. It includes for loop, while loop and do while loop.

@Er_BHISMARAJ

5. Object Oriented Programming Techniques

- It is a modern approach of programming in which both data and function are combined into a single unit which is called object.
- This approach is based on the data rather than procedure (function) and programs are normally divided into objects and class.

- When the programmer is coding by using the modern programming languages like Java, Python C++, dot net etc. we can say he is following this OOP approach.
- The key concepts of OOP are encapsulation, inheritance, polymorphism, and abstraction. Here’s a detailed explanation with an example:

Compiled by Er BRJ

@Er_BHISMARAJ

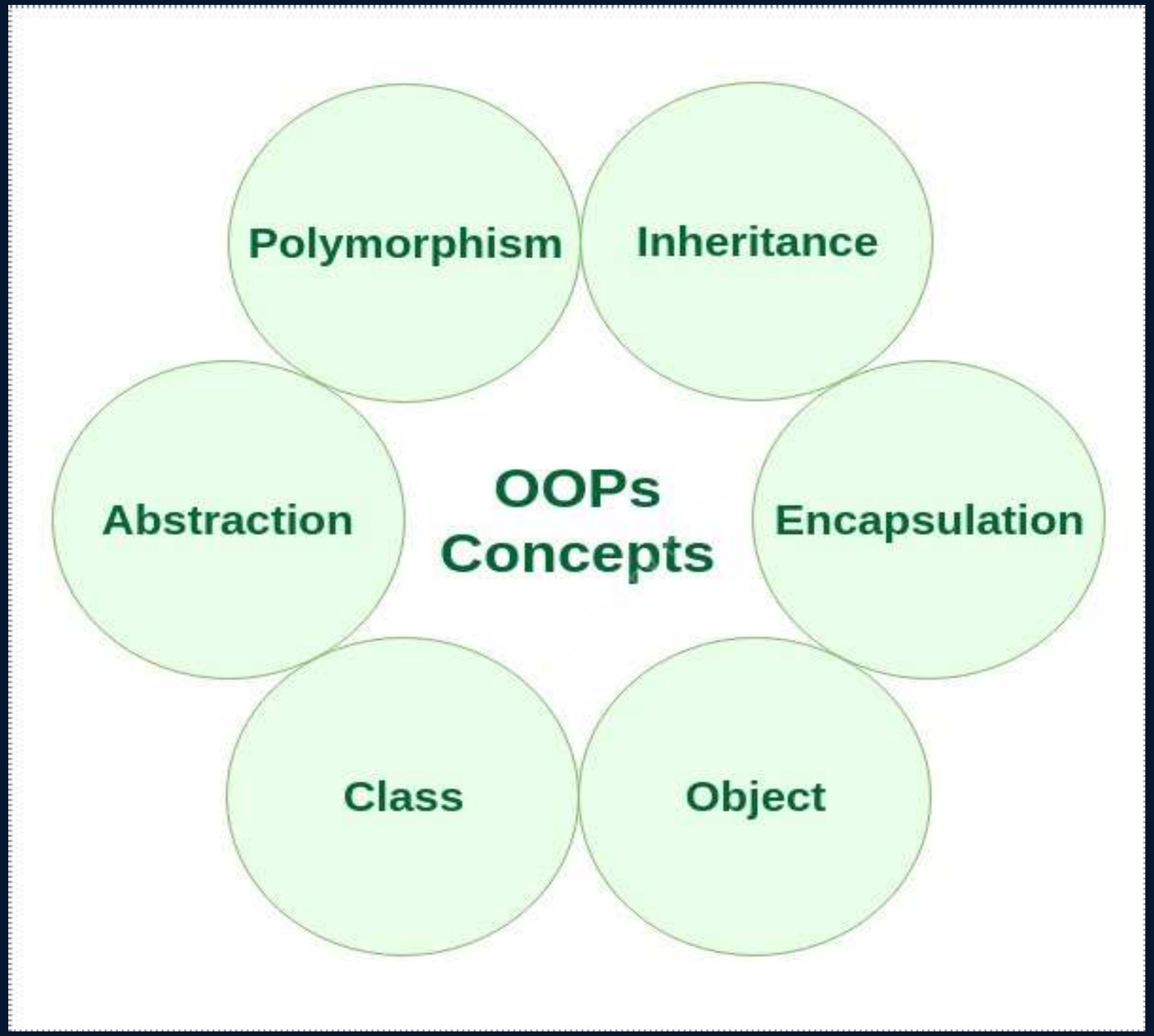


Fig:- Object Oriented Programming Concept

@Er_BHISMARAJ

ASSIGNMENT-3

WRITE DOWN THE MAJOR ADVANTAGES AND DISADVANTAGES OF STRUCTURED PROGRAMMING APPROACH AND OBJECT-ORIENTED PROGRAMMING APPROACH.

Compiled by Er BRJ

@Er_BHISMARAJ

PROGRAM (SOFTWARE) DEVELOPMENT MODELS

- A software or program development model is a structured and systematic approach that describes the activities like planning, designing, building, testing and deploying of software system.

- It serves as a framework that outlines the tasks, activities and processes involved in developing the software.

- These models provide the guidelines and best practices for software development team through out the project life cycle.

Compiled by Er BRS

@Er_BHISMARAJ

Mostly used models for program development are:-

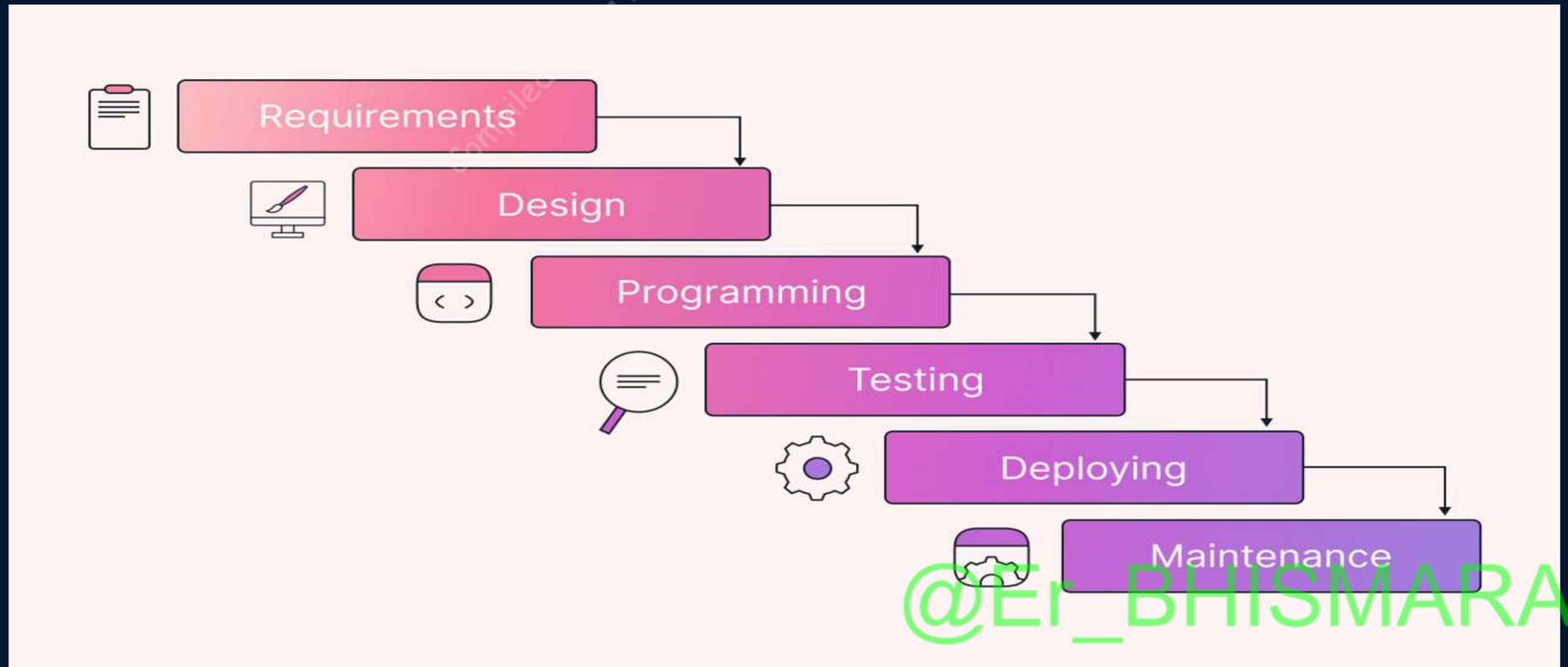
1. Waterfall Model
2. Prototype Model
3. Agile Model or Methodology
4. Spiral Model
5. Iterative Model
6. V-Model
7. RAD Model

Compiled by Er BHI

@Er_BHISMARAJ

1.Waterfall Model

- This model was originally published in 1970 by Winston W. Royce.
- It is a linear and sequential approach where each phase is completed before moving to the next phase. It is the one of the oldest type of software development model.
- The diagrammatic representation of waterfall model is shown in figure below.



@Er_BHISMARAJ

i) Requirement Analysis

- It is the first state of waterfall model. In this stage the developer should identify the actual requirements of the customer.

ii) System Design

- Once the requirement of the customers have been collected, the design phase is conducted. In this phase the team build out the skeleton of how the software will work and how information will be accessed.

iii) Programming/Development

- During the programming phase, the system design is converted into the actual product by programming or coding. This development phase involves the creation of the system software.

Compiled by ER BJ

@Er_BHISMARAJ

iv) Testing

- This is the phase where the development teams hands the project over the quality assurance or testing team.QA testers search for any bugs or errors that need to be fixed before the project is deployed.

v) Deployment

- This is the stage in which the ready software product is deployed to the end users or customers.

Compiled by Er BRJ

vi) Maintenance

- Once the product is deployed, there may be chances of discovering the new bugs on the project. Which are required to be maintain so that it can works as expectation.

(Assignment-pros & cons of waterfall model.)

@Er_BHISMARAJ

2) Prototyping Model

- The software development model that focuses on creating a working model, called prototype of the software to gather the feedback and refine the requirements before developing the final product.

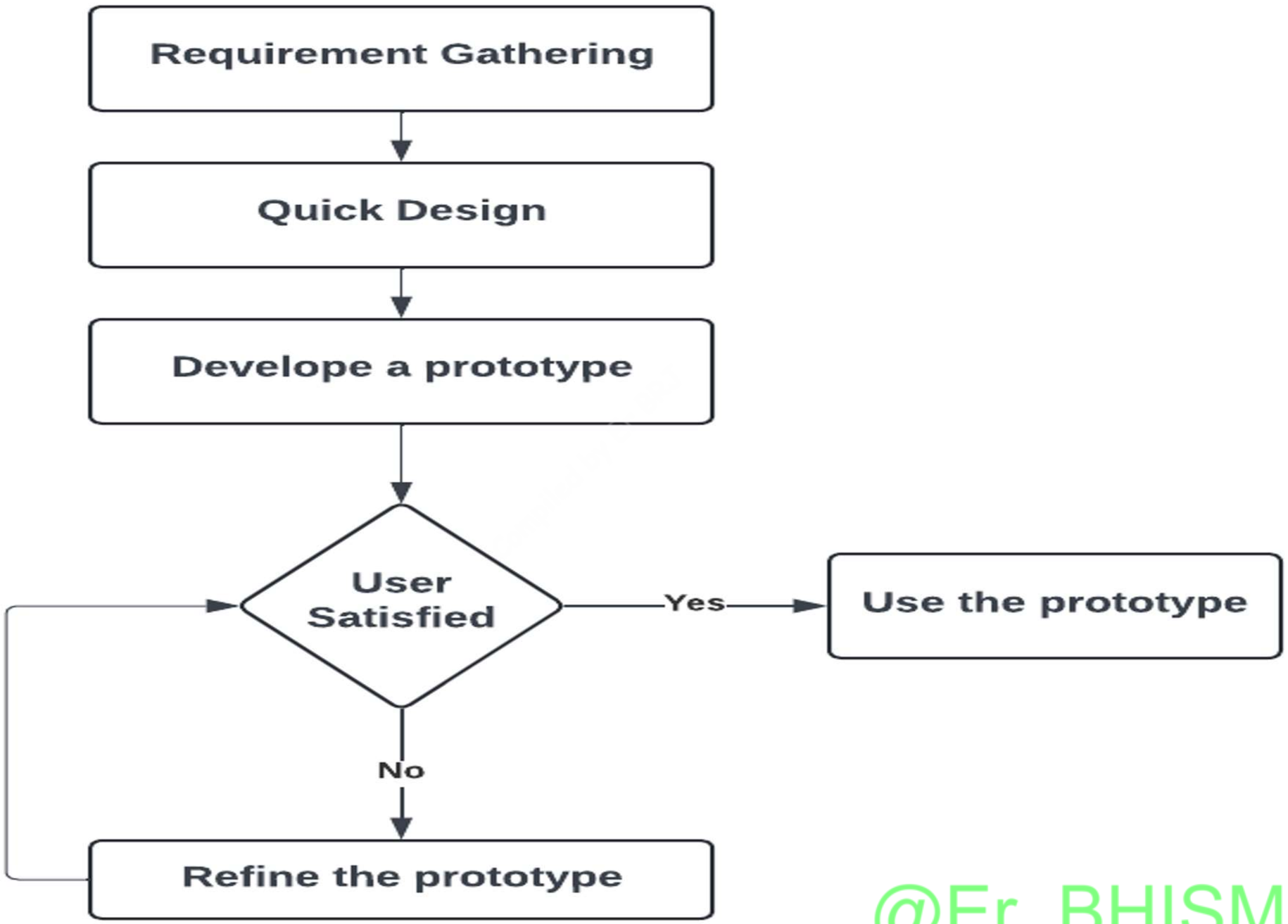
- This model is more suitable for developing the new system where there is no clear ideas of requirements, inputs and output of the system.

- The diagrammatic representation of this model is shown in fig below.

Compiled by Er BRJ

@Er_BHISMARAJ

Fig :- Prototyping Model



@Er_BHISMARAJ

i) Requirement Gathering

- In this phase the initial requirements are collected from users but these are mostly incomplete.

ii) Quick Design

- Once the requirement phase is completed, the basic design of the software is created by using different software design tools.

iii) Develop the prototype

- The prototype is developed based on the initial design. The prototype can take various form.

- Throwaway prototyping:- A disposable prototype is developed just for understand the requirement. Once its purpose is served, then it is discarded.
- Evolutionary prototyping:- The initial prototype is refined continuously and evolved into the final product.

Compiled by Er BRJ

@Er_BHISMARAJ

iv) User satisfaction

- The prototype is presented to the end users for evaluation and feedback. This phase is very important to clarifying the requirements.

v) Refinement

- Based on the feedback received the prototype and requirements are refined. In this phase the new features and changes are added into the design.

vi) Deployment

- Once the system is developed it is deployed on the production as a finished product.

Compiled By Er BRJ

@Er_BHISMARAJ

Advantages of prototyping model

- Errors can be detected earlier.
- Higher chances of user expectation meet.
- Suitable where user requirements are changing

Dis-advantages of prototyping model

- Time consuming process.
- Special tools and techniques are required.
- It may increase the complexity of the system.

Compiled by Er BRJ

@Er_BHISMARAJ

3) Agile model / methodology

- An agile methodology is an iterative approach of software development. It breaks down the development of a product into repeated cycles (iterations).

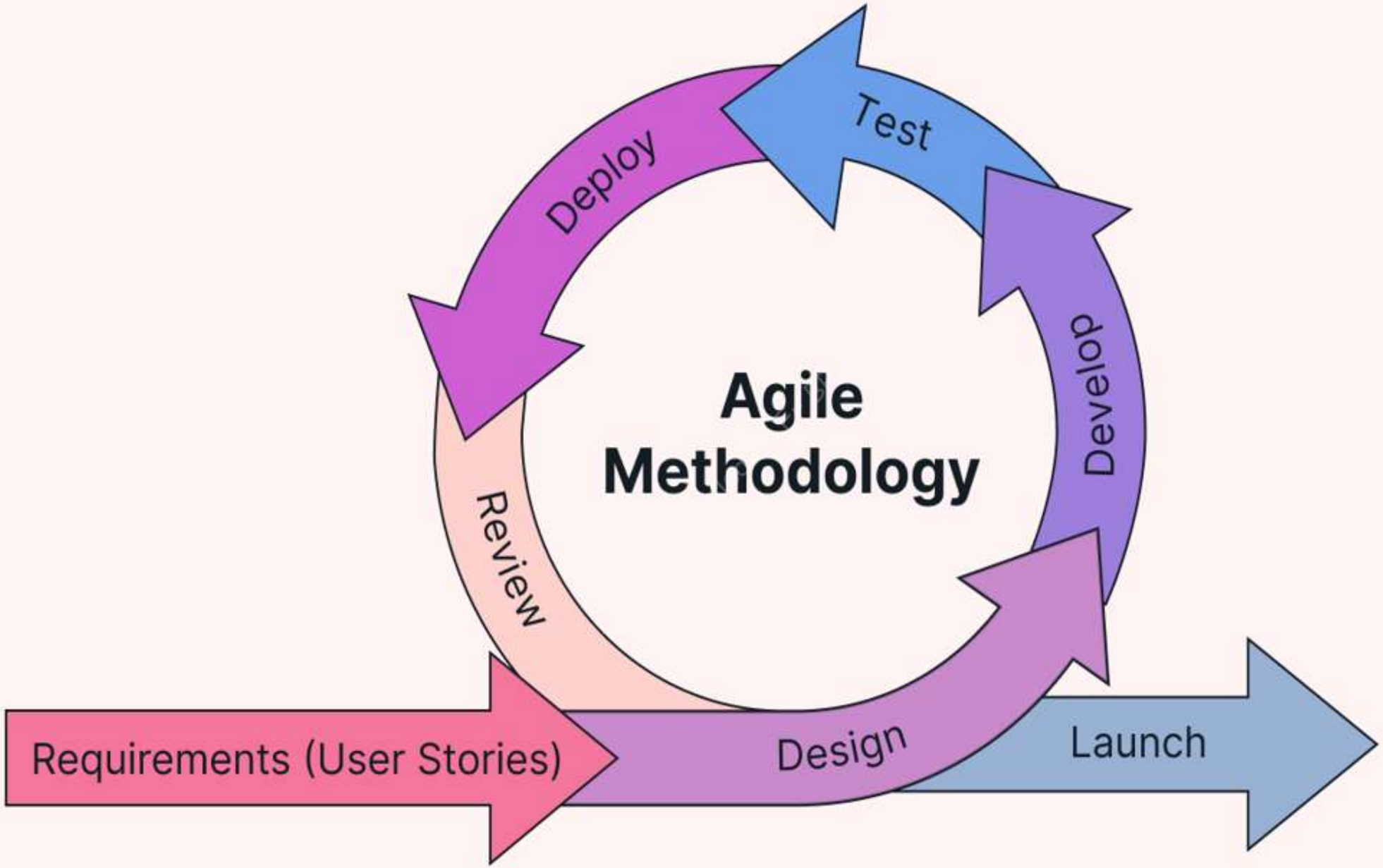
- It refers to a set of principles and practices for software development that focus on flexibility, collaboration and customer satisfaction.

- The diagrammatic representation and the different phases involved in agile methodology are as follows.

Compiled by Er BRJ

@Er_BHISMARAJ

Fig:-Agile methodology



@Er_BHISMARAJ

i) Requirement Gathering

- This is the first phase in which the basic requirements of the customers are gathered and plan for the further steps.

ii) Design

- When all the requirements are identified, the design phase is conducted where the different design tools like ER diagram, UML diagram and data flow diagram are used for the system design.

iii) Development

- When the design phase is completed the development phase is conducted. In this phase the system design is provided to development team and according to the provided design, the developer team start develop the project.

Compiled by ER BHIS

@Er_BHISMARAJ

iv) Testing:- Tester team simply try to find out the bugs and any errors present in the system. If errors are find-out then its fixed.

v) Deployment:- After the successful testing the ready product is deployed on the cloud or handover to the customer.

vi) Review and Feedback:- After releasing the product, the team receives the feedback about the product and their works. According to the feedback the team can find out the customer satisfaction.

vii) Launch:- It ensures that the software is successfully deployed, configured, and accessible to users. It marks the transition from development to real-world usage.

Compiled by Er BRJ

@Er_BHISMARAJ

ASSIGNMENT:-4

EXPLAIN AGILE METHODOLOGY IN DETAIL WITH NEAT AND CLEAN DIAGRAM. WRITE DOWN ITS ADVANTAGES AND DISADVANTAGE.

Created by @BRJ

@Er_BHISMARAJ

4. Spiral Model

- The Spiral model is published in 1998 by Barry Bohem in 1986.
- This software development model combines the elements of both waterfall model and prototyping model is called spiral model.
- This model is particularly well suited for the large scale, expensive and complex projects.
- The focus of this model is on risk management throughout the entire software development life cycle.

Compiled by Er BHI

@Er_BHISMARAJ

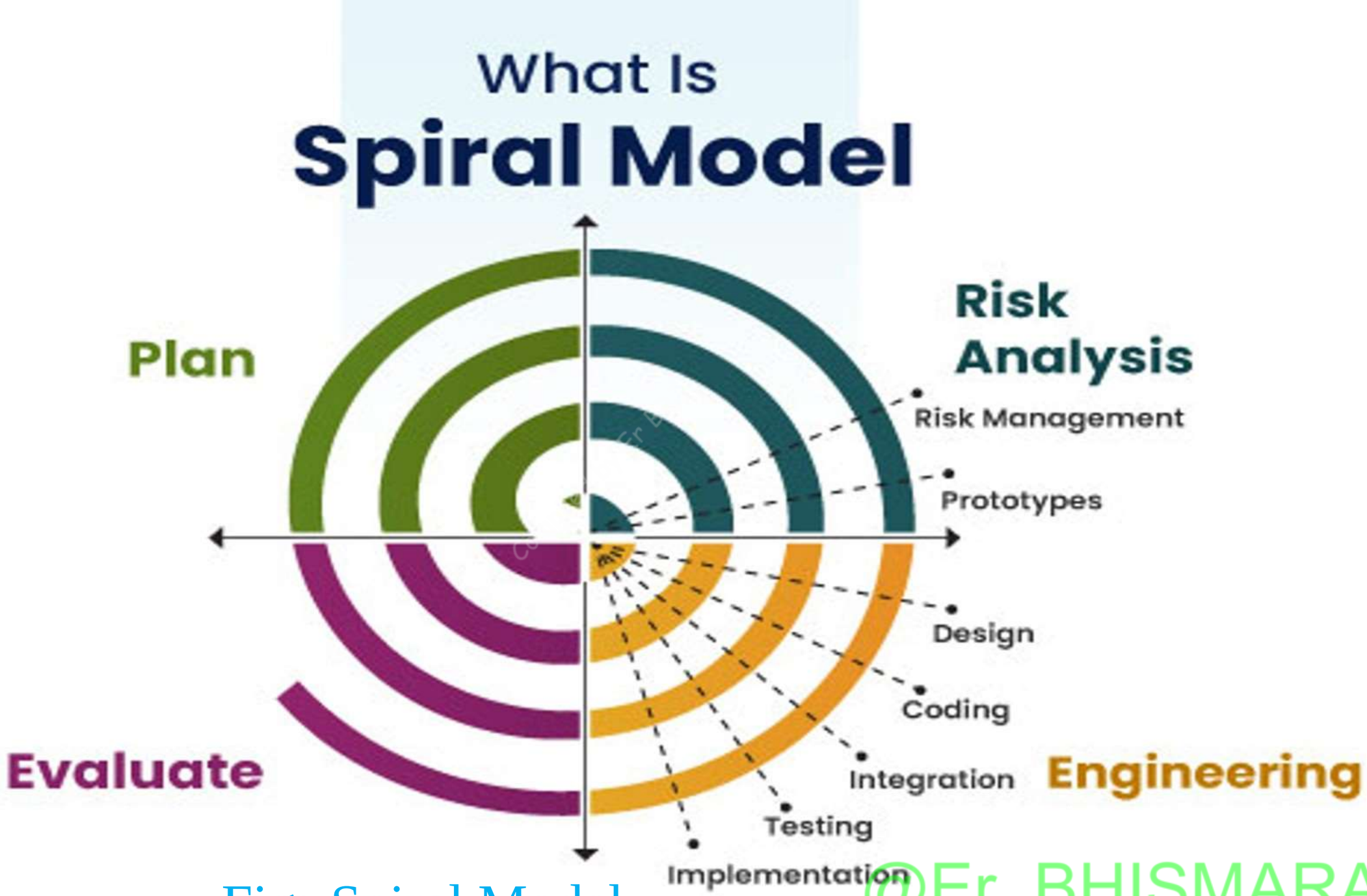


Fig:-Spiral Model

@Er_BHISMARAJ

-The different phases involved are:-

1) Planning

- In this initial phase, project objectives, requirements are defined and decide to continue further loop of spiral. If decided to continue, plans are drawn up for the next phase.

2) Risk Analysis

- In this phase the developer team will identify all the potential risks like requirement changes, security vulnerabilities, poor planning, scheduling issues, team turnover, market changes, budget overruns, stakeholder conflicts etc. that will arise during the software development process.
- In short, this phase is all about risk mitigation and risk management.

Compiled by Er BRJ
@Er_BHIGMARAJ

3) Engineering:- In this phase the actual development of the product or software occurs. This phase is similar to the development phase where the programmers write down the codes for the project.

- In this phase different tasks like design, coding, integration, testing and implementation are performed.

Compiled by Er BRJ

4) Evaluation:- After the launch of a software project on the client's site, the client is open to sharing feedback about the software.

If there are any bugs or errors are found, then the correction lists will be sent to the development team, and again, a new baseline will begin to fix all the bugs, and after fixation of all requirements, the final product will be ready for final launch.

@Er_BHISMARAJ

5. Iterative Model

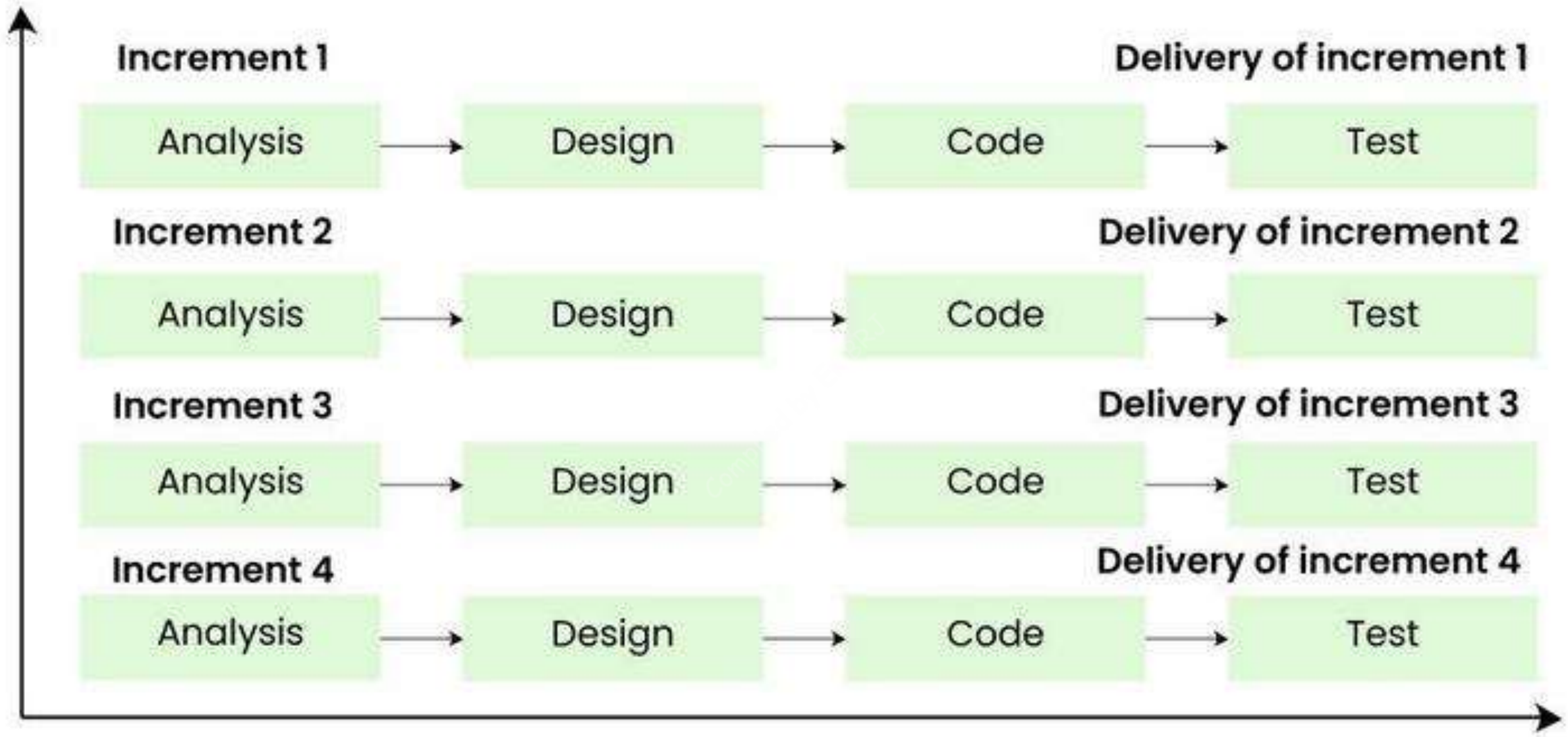
- Simply iteration refers to the process of repeating a set of instructions or steps in a program multiple times. This repetition continues until a particular condition is met.

- The iterative model is a software development approach that involves the repetitive process of refining and improving the software through multiple iterations or cycles.

- Each iteration produces a working version of the software, which is then evaluated, tested, and enhanced in subsequent iterations.

Compiled By Er BRJ

@Er_BHISMARAJ



Life-Cycle of Iterative Enhancement Model

@Er_BHISMARAJ

- The different phases of iterative models are:-

i) Requirements Analysis:

- Gather the different requirements of the clients by using different requirement gathering methods and analyze requirements for the entire system. So that it can be easy to identify core functionalities.

ii) Design:

- In this phase the designer team create high-level and detailed design for the on-going product. This includes designing the architecture, interfaces and database schemas, that guide the implementation of the product.

iii) Coding / Implementation:

- Write the code and develop the software components for the current project.

Compiled by Er BRJ

@Er_BHISMARAJ

iv) Testing:

- Test the product to ensure it meets the specified requirements and functions correctly or not. This includes unit testing and integration testing.

v) Deployment and Feedback:

- Deploy the completed product to its users and gather the feedback from the users after deployment.

vi) Planning for Next Increment:

- Plan for the next increment based on the feedback received and remaining requirements. Update the project plan and schedule for the next cycle.

Compiled by Er BRJ

@Er_BHISMARAJ

6. V-Model

- The V-Model, also known as the Verification and Validation Model, is an extension of the traditional Waterfall model and it is based on a parallel relationship between development stages and corresponding testing stages, forming a V-shape.

- This model ensures that for every development activity, there is a corresponding testing activity.

- In this model verification refers, ensuring that the product is built correctly and validation ensures that the correct product is built.
- Here's a detailed explanation of the V-Model:

Compiled by Er BRJ

@Er_BHISMARAJ

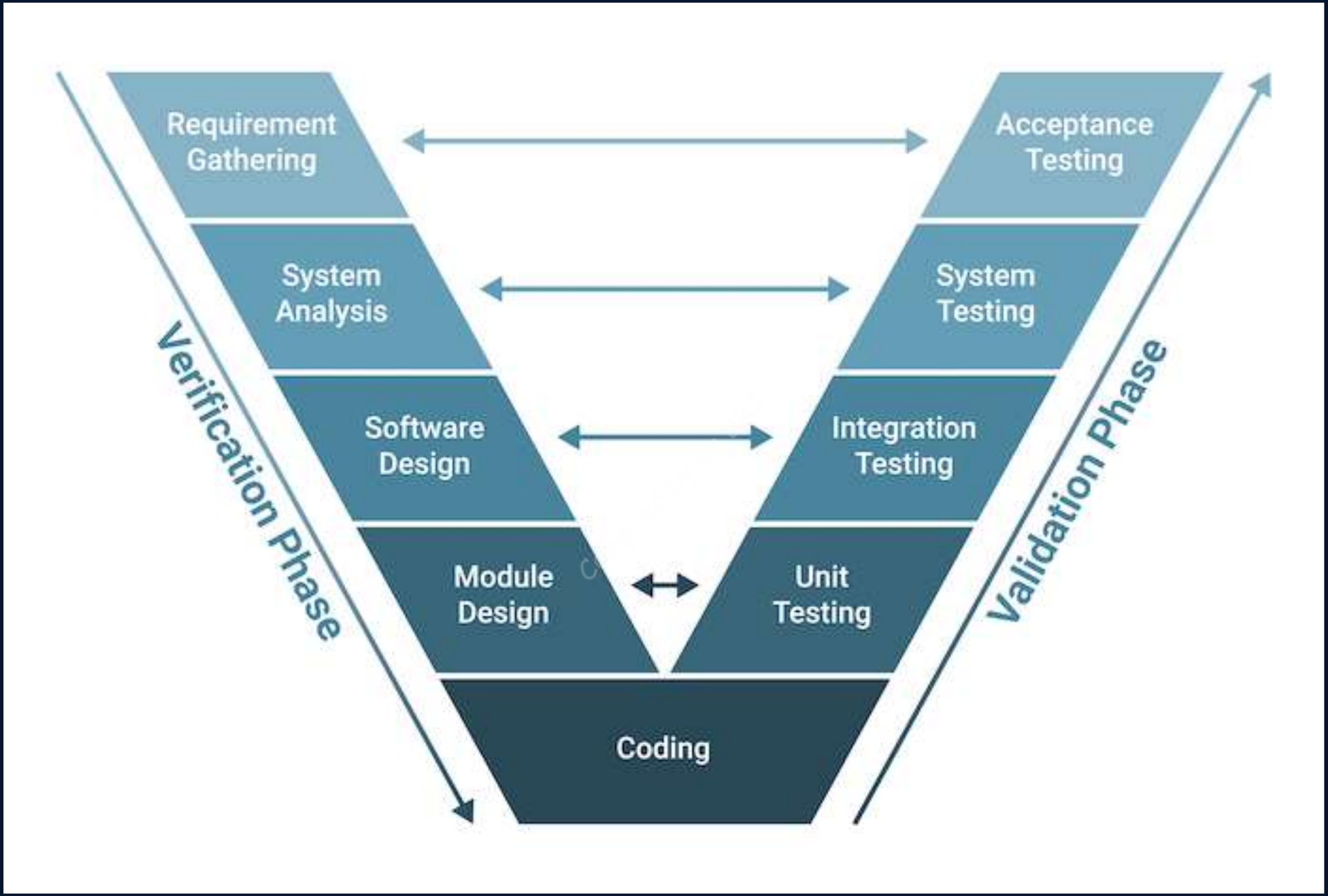


Fig:- V-Model

@Er_BHISMARAJ

Verification Phases:

1.Requirement Gathering Analysis:- Gather and analyze user requirements and prepare a requirements specification document.

2.System Design:- Define the overall system architecture and System design documents.

3.Software Design:- Define the system's modules, components, and their interactions. So that the high-level design documents can be prepared.

4.Module Design:- Define the internal design of each module/component and prepare the low-level design documents.

5. Implementation / Coding:- Write the code and implement the design to obtain the final product.

Compiled by Er BRJ

@Er_BHISMARAJ

Validation Phases:

1.Unit Testing:- Test individual modules/components for correctness and this testing is performed by Developers.

2.Integration Testing:- Test the interaction between integrated modules/components of the products. Performed By: Developers or independent testers(which are not involved in development process).

3.System Testing:- Test the complete system for compliance with the specified requirements. Performed By: Independent testers.

4.Acceptance Testing:- Validate the system against user requirements and ensure it meets their needs or not. And this testing is performed by: End-users or clients.

Compiled by ER BRJ

@Er_BHISMARAJ

Write down the advantages and dis-advantages of spiral, iterative and V model .

Compiled by Er_BHISMARAJ

@Er_BHISMARAJ

7. RAD Model

- RAD stands for Rapid Application Development model. It is an incremental software development process model that focus on extremely short development cycle.

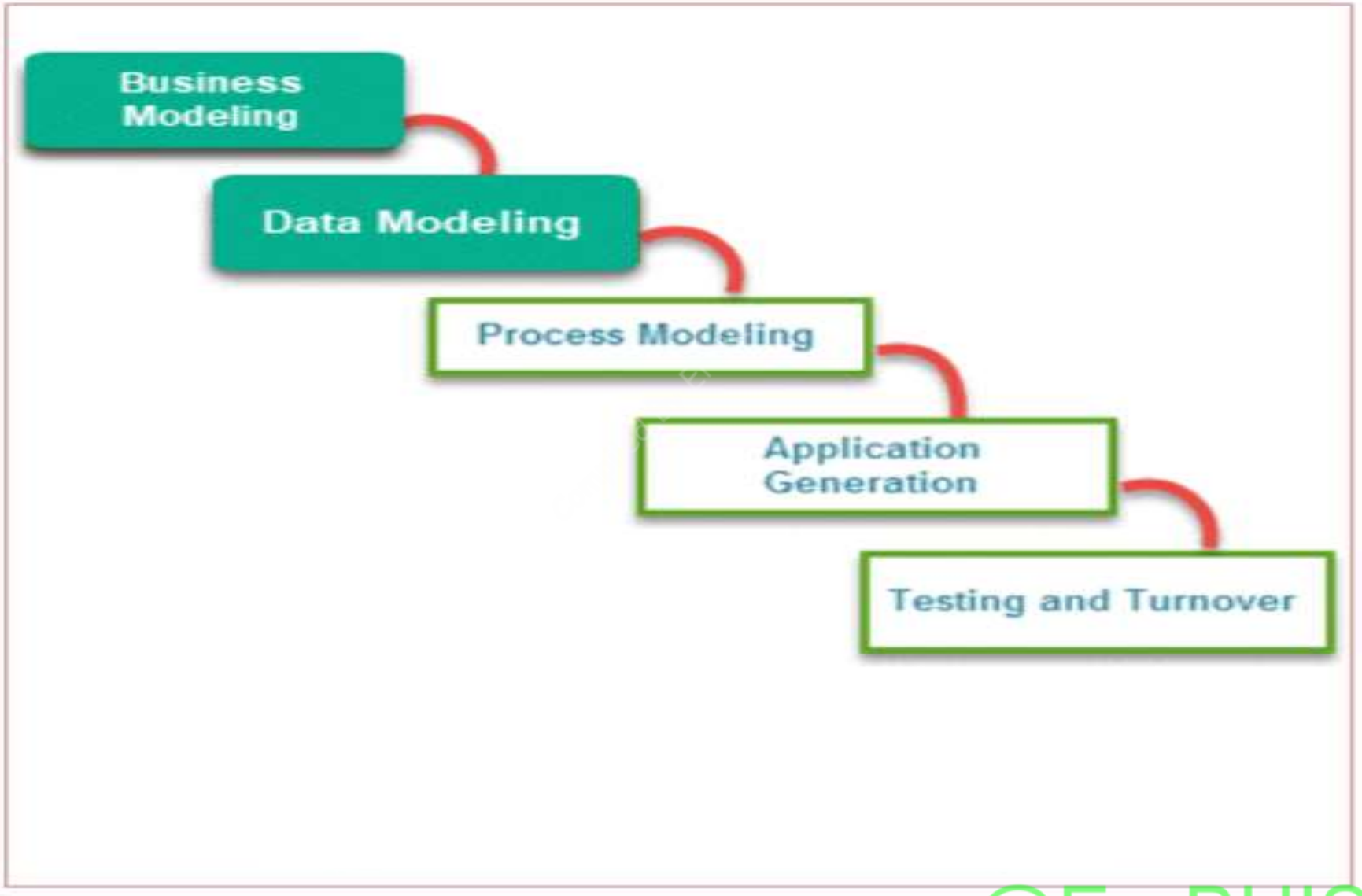
- It is a high speed adaptation of linear sequential model in which development is achieved by using component based construction.

- It is suitable when users requirements are well-known and that require a fast turnaround time.

Compiled by Er BKR

@Er_BHISMARAJ

- The diagrammatic representation of RAD model is shown in figure below.



RAD Model Diagram

@Er_BHISMARAJ

The different phases of RAD model are

i) Business Modeling

- It refers to the process of understanding and defining the core business processes, goals, and objectives that the application or system is designed to address.
- It plays a crucial role in ensuring that the application being developed is aligned with the business needs and delivers value to the organization.

ii) Data Modelling

- It refers to the process of creating a conceptual representation of the data structures that the application will use.
- In RAD, data modelling is done quickly, typically in the early stages of the development cycle, and focuses on the essential data entities, relationships that support the business processes that the system is designed to address.

Compiled by Er. BRJ

@Er_BHISMARAJ

iii) Process Modelling

- The data objects defined in the data modelling phase are transformed to achieve the data flow necessary to implement a business function.
- Processing descriptions are created for adding, modifying, deleting, or retrieving a data object.

iv) Application Generation

- The Application Generation phase in the RAD model uses automated tools and fourth-generation languages (4GL) to rapidly develop software through prototyping, continuous user feedback, and integration.
- This phase emphasizes quick adjustments, component integration, which helps to meets user needs and ensures functionality in the production environment.

Compiled by Er BRJ

@Er_BHISMARAJ

v) Testing and Turnover

- The Testing and Turnover phase in the RAD model is crucial for ensuring the final product is robust and meets user requirements.
- This phase involves testing like unit testing, integration testing, system testing, and user acceptance testing.
- Once testing is complete and any issues are resolved, the application is deployed to the production environment. User training and documentation are also provided during this phase.

Compiled by Er BRJ

@Er_BHISMARAJ

Assignment

Write short note on Big Bang model.

Compiled by BRJ

@Er_BHISMARAJ

Cohesion and Coupling

- Cohesion in software engineering refers to the degree to which the elements of a module or component are functionally related to each other. It represents how tightly bounded the internal elements of the module with one to another.

- High cohesion within a module means that its elements work together towards a single, well-defined task.

- Cohesion is essential for creating maintainable, robust, and understandable code. It makes the code easier to modify and reduces the chances of errors.

Compiled by Er BRJ

@Er_BHISMARAJ

Types of Cohesion

- Cohesion is classified into different types, typically in increasing order of quality (from low to high cohesion):

1. Coincidental Cohesion

- It is the lowest level of cohesion and in this type of cohesion the different elements have no meaningful relationship with each other.
- Example: A utility module that contains unrelated functions like logging, string manipulation, and file operations, date and time utilities.

2. Logical Cohesion

- Logical cohesion is when parts of module are grouped because they are logically related to do the same thing but not functionally.
- Example: Input Handling Module, that handles all user input types, such as keyboard, mouse, gesture input.

@Er_BHISMARAJ

iii) Temporal Cohesion

- Temporal cohesion occurs when the elements of a module are grouped because they are executed at the same point in time, such as during initialization, cleanup, or shutdown processes.

- Suppose if you are developing a system where multiple resources need to be initialized when the application starts, such as:

- Opening a database connection
- Loading configuration settings
- Setting up logging

These tasks are grouped in a single module, executed together during application startup.

Compiled by Er BHI

@Er_BHISMARAJ

4. Procedural Cohesion

- Procedural cohesion occurs when elements of a module are grouped because they need to be executed in a specific order to achieve a task.
- The tasks are functionally related but are connected primarily by the sequence of execution.

Example user login process (Accepting user's credentials, Validating the credentials, Start a session)

Compiled by Er BRJ

5. Communicational Cohesion

- Components operate on the same data or contribute to the same output.

Example: A module that processes and validates user input before storing it in a database.

@Er_BHISMARAJ

6. Sequential Cohesion:

- Elements are related in such a way that the output of one element is the input of the next.

Example: A module that reads data from a file, processes it, and writes the result to another file.

7. Functional Cohesion

- **It** is the highest and most desirable level of cohesion in software engineering.
- It occurs when all the elements within a module are related by their contribution to a single, well-defined task.
- Example A module that calculates the sum of an array, performs matrix multiplication, or computes the factorial of a number.

Compiled by Er BRJ
@Er_BHIMARAJ

Coupling

- “Coupling" refers to the degree of interdependence between software modules and how closely connected they are.
- It measures how much one module relies on another. High coupling means that changes in one module will likely require changes in another, which can make the system more difficult to understand and maintain.
- For good software, low coupling is generally preferred. Low coupling means that each module or component of the software operates independently of the others. Which helps in easier maintenance, Improve Modularity and better scalability.

Compiled By Er BRS

@Er_BHISMARAJ

Types of Coupling

1. Content Coupling

- It is the type of coupling in which one module can directly access or modifies the data or logic of another module.
- It is the most tightly coupled and least desirable in software development.
- Example Module A, directly change a variable in Module B.

Compiled by Er BRJ

2. Common Coupling

- It is also known as global coupling in this coupling, the modules share a common global data or variable for example: Two modules accessing and modifying the same global variable.
- The debugging becomes difficult since changes in one module can affect others using the same global data.

@Er_BHISMARAJ

3. Control Coupling

- The one module controls the behaviour of another by passing information like control flags for what to do. It reduces module independence and makes the code less readable and harder to maintain.

Ex: Module A passes a control flag to module B, and control B's flow.

4. Stamp Coupling (Data-Structured Coupling):

- In this coupling mechanism one modules share a composite data structure and another module use only a part of it.
- It leads to unnecessary interdependence and complexity, as changes to the shared data structure may require changes to all modules using it.
- Example: Module A passes a whole record to module B, but B only uses a part of that record.

Compiled By Er BRJ

@Er_BHISMARAJ

5. Data Coupling

- Data coupling occurs when modules interact by sharing data exclusively through parameters.
- This form of coupling is ideal in software design because it minimizes interdependencies between modules, allowing them to function independently and making the system more modular and easier to maintain. By sharing only the necessary data, data coupling promotes flexibility, scalability, and easier debugging.

6. Message Coupling

- Modules interact via message passing, usually through interfaces or APIs. It promotes loose coupling and high cohesion as modules interact without knowing each other's internal workings.

Example: Module A sends a message to module B via an interface.

@Er_BHISMARAJ

ASSIGNMENT

WHY COHESION AND COUPLING ARE REQUIRED? EXPLAIN IN DETAILS.

Compiled by Er BRJ

@Er_BHISMARAJ

Communication Between Modules

- As we know in modular programming, a program is divided into separate modules or components, each responsible for a specific piece of functionality.
- To continuous the programming process every modules have to communicate with each other i.e. the programmer designs the modules with the connection with the each other.
- In other word one module calls another module to complete the specific task and while calling a module data is passed to the procedure.
- Communication between these modules is essential for building well-structured and maintainable software systems.

Compiled by ER BRJ

@Er_BHISMARAJ

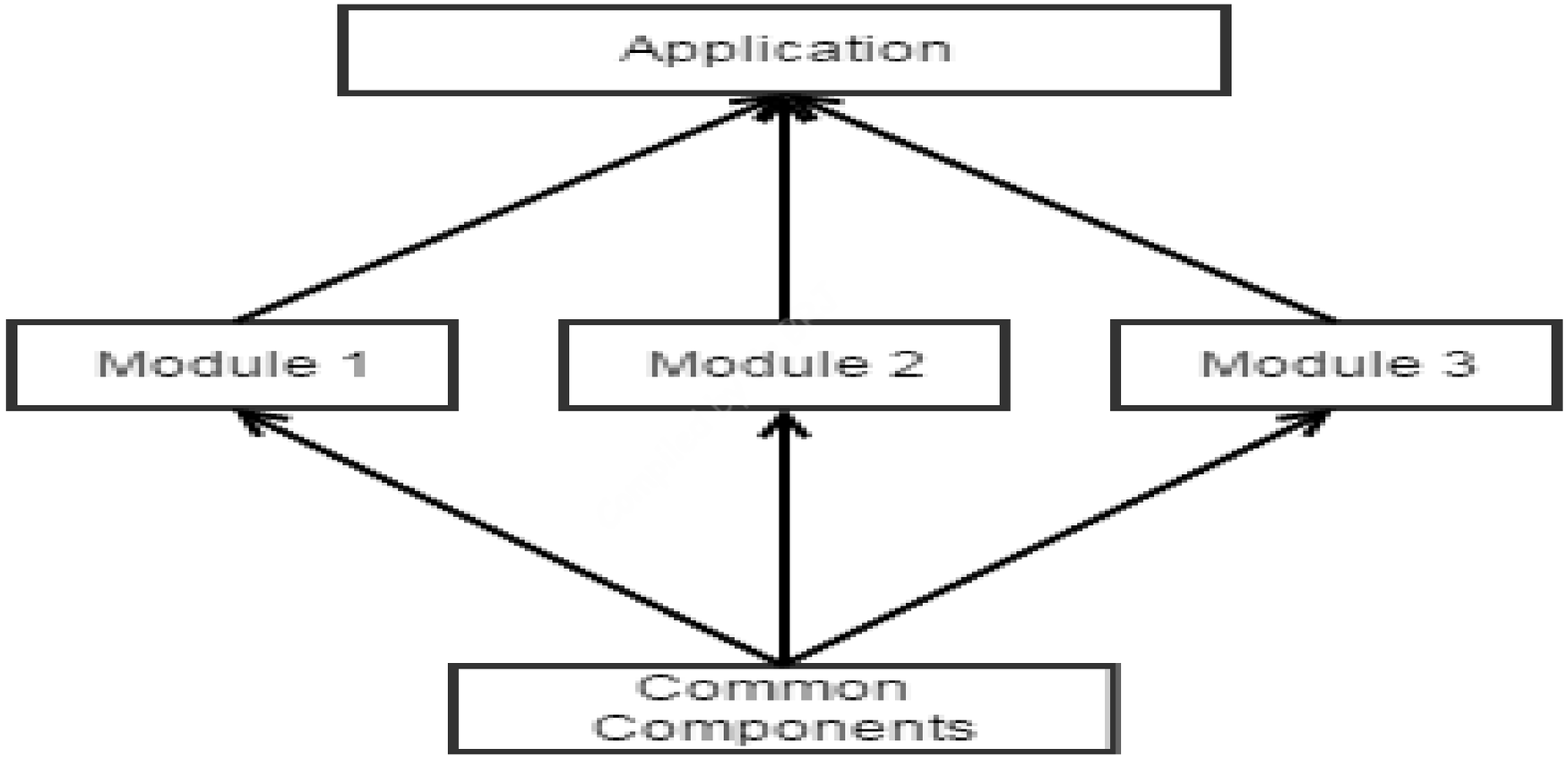


Fig:- Communication between modules

@Er_BHISMARAJ

- About the inter-module process there are two types of information passed between modules which listed below:

1. Data
2. Control

- The data pass from the main module to the lower level module, to call the lower level module.
- Generally, the first module is main module and it said to call or invoke the lower level module. In the Root module data, parameter or address can be passed to the lower level module.

- Similarly, the root module passes the control down at the lower level modules. The control is always returned to the invoking module and as a finished module should always terminate at the root.

Compiled by Er BRJ
@Er_BHISMARAJ

Types of Communication:

- Direct Function Calls: One module can call functions or methods of another module directly, allowing for straightforward interaction.
- Shared Data Structures: Modules can share common data structures, such as global variables or data passed through parameters.
- Message Passing: Modules can send messages to each other, often used in concurrent or distributed systems.
- Event-Driven Communication: Modules can communicate by emitting and listening to events, commonly seen in GUI applications and asynchronous programming.

Compiled by Er BRJ

@Er_BHISMARAJ

Logics of Program Development

- Programming logic is the foundation of any software development process. It involves using a systematic and structured approach to solve problems using a computer. Here are some key logics commonly used in programming:
- The basic logical structures are

i) Sequential Structure

- A sequential structure refers to the simplest control flow structure where statements are executed one after another.

```
<START>
STATEMENT1;
STATEMENT2;
STATEMENT3;
<STOP>
```

Compiled by Er BRJ

@Er_BHISMARAJ

ii) Selection Structure

- Which is use to make decisions based on the several conditions.
- It consists of two parameter: condition and statement

Types of Selection Structures in C:

1. if statement
2. if-else statement
3. else if ladder
4. nested if statement

Compiled by Er BRJ

@Er_BHISMARAJ

1. if statement

- The simplest form of selection structure is the if statement. It evaluates a condition (which is a logical expression) and executes a block of code if the condition is true. Syntax

```
if (condition) {
   // code to be execute if }
```

2. if-else statement

- The if-else statement provides two paths: one if the condition is true and another if the condition is false. Syntax

```
if (condition) {
        // Block of code to execute if the condition is true
       } else {
          // Block of code to execute if the condition is false }
```

Compiled by Er BRJ

@Er_BHISMARAJ

3. else if ladder

- The else if ladder is used when multiple conditions need to be checked. If the first condition is false, the next condition is evaluated, and so on.
- If none of the conditions are true, an optional else block can be used as a default case.

```
if (condition1) {
    // code to execute
} else if (condition2) {
    // code to execute
} else {
    // code to execute
}
```

Compiled by Er BRJ

@Er_BHISMARAJ

4. nested if statement

- A nested if is an if statement placed inside another if or else block. This allows for checking multiple conditions in a hierarchical manner.

Syntax

```
if (condition1) {
    if (condition2) {
        // code to execute if both conditions are true } } example
#include <stdio.h>
int main() {
    int num = 5;
    if (num > 0) {
        if (num % 2 == 0) {
            printf("Number is positive and even\n"); } else {
            printf("Number is positive and odd\n"); } }
    return 0; }
```

Compiled by Er BRJ

@Er_BHISMARAJ

iii) Iterative(Looping) Structure

- In C programming, a looping structure (or iterative structure) allows you to execute a block of code repeatedly until a certain condition is met, otherwise the program is terminated.

There are mainly two types of loops in C Programming:

1) Entry Controlled loops: In Entry controlled loops the test condition is checked before entering the main body of the loop. For Loop and While Loop is Entry-controlled loops.

2) Exit Controlled loops: In Exit controlled loops the test condition is evaluated at the end of the loop body. The loop body will execute at least once, irrespective of whether the condition is true or false. do-while Loop is Exit Controlled loop.

Compiled by Er_BHI

@Er_BHISMARAJ

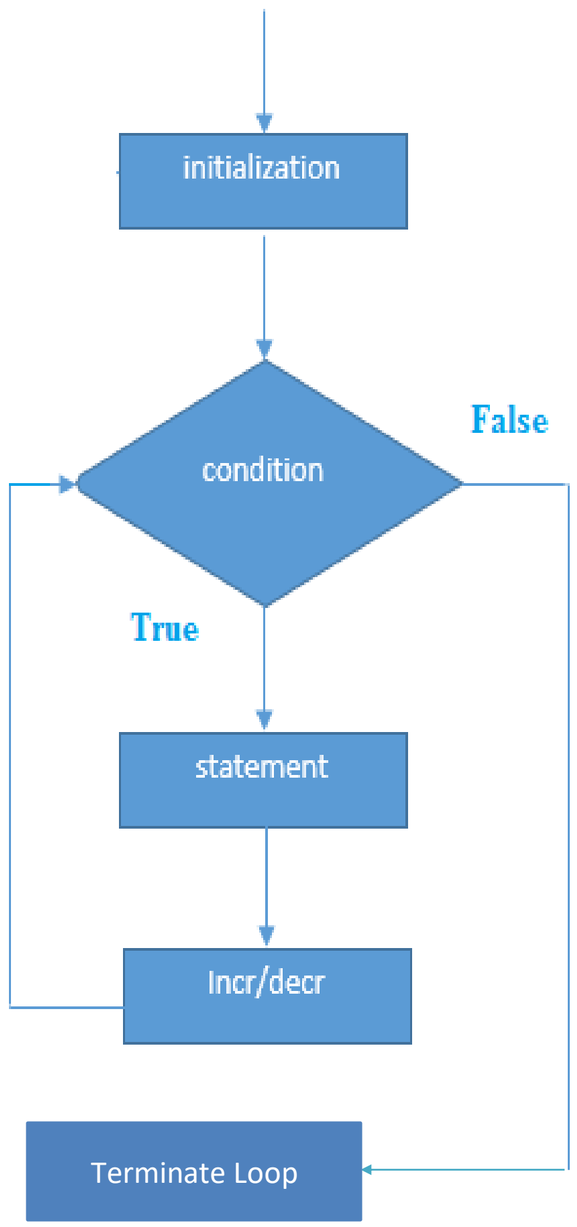
a) For Loop:- The looping structure in which the statement is executed at a specific number of times repeatedly if the condition is true otherwise the program is terminated.

- It belongs to the entry control loop.

Syntax

```
for (initialization; condition; update) {
   // Block of code to execute }
```

fig:- Flow Diagram for for loop:



Compiled by Er BRJ
@Er_BHISMARAJ

b) While Loop

- The while loop in C programming is a control flow structure that allows you to execute a block of code repeatedly as long as a specified condition remains true.
- It belongs to the entry-controlled loop category, meaning that the condition is checked before the loop body is executed.

Syntax

```
while (condition) {
 // block of code to execute }
```

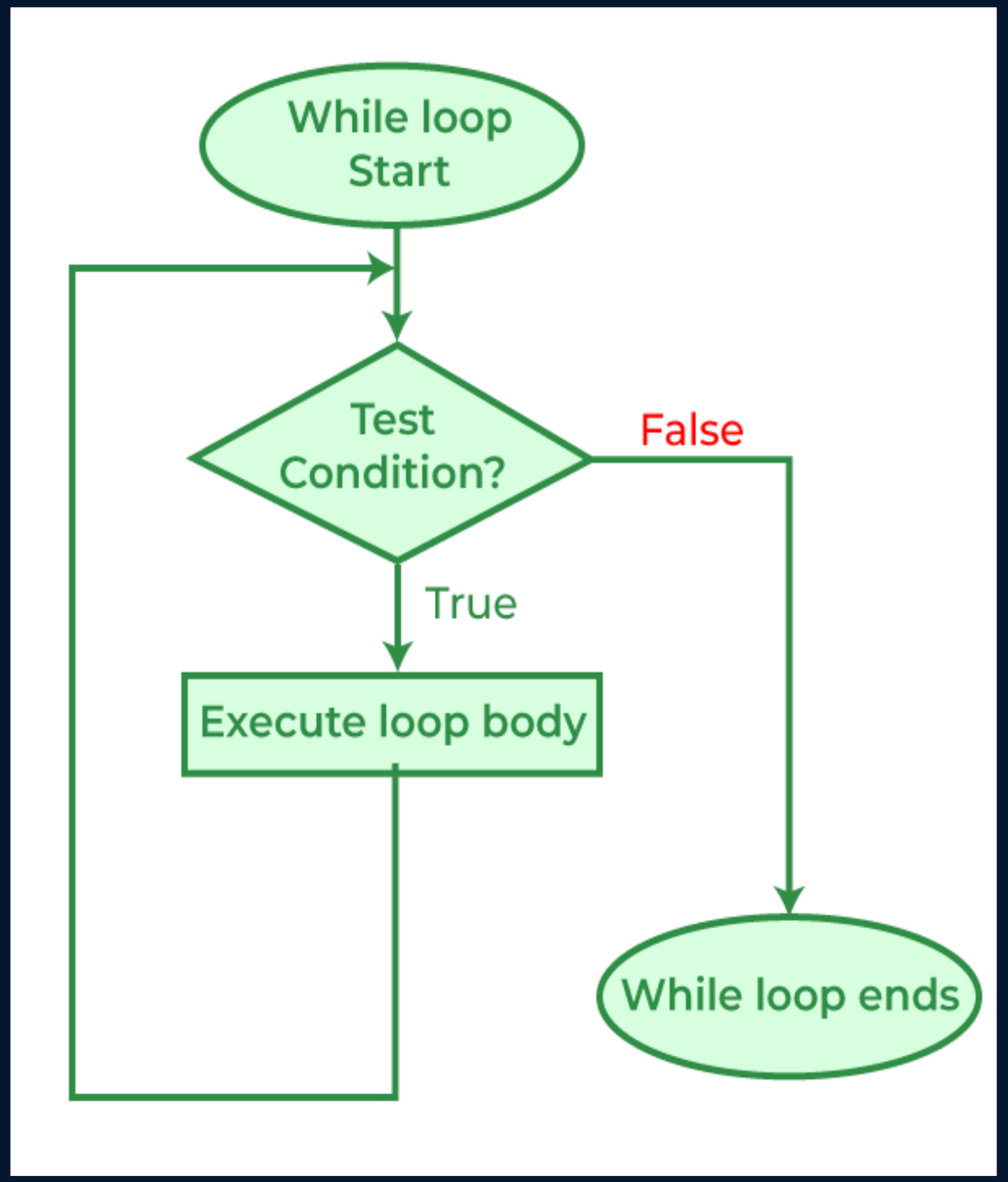


Fig:-Flow diagram for while loop.

@Er_BHISMARAJ

c) Do while loop

- The do-while loop in C programming is an exit-controlled loop, meaning that the condition is evaluated after the loop body has been executed. This ensures that the loop body is executed at least once, even if the condition is initially false.

Compiled by Er BRS

Syntax

```
do {
   // code to execute)
} while (condition);
```

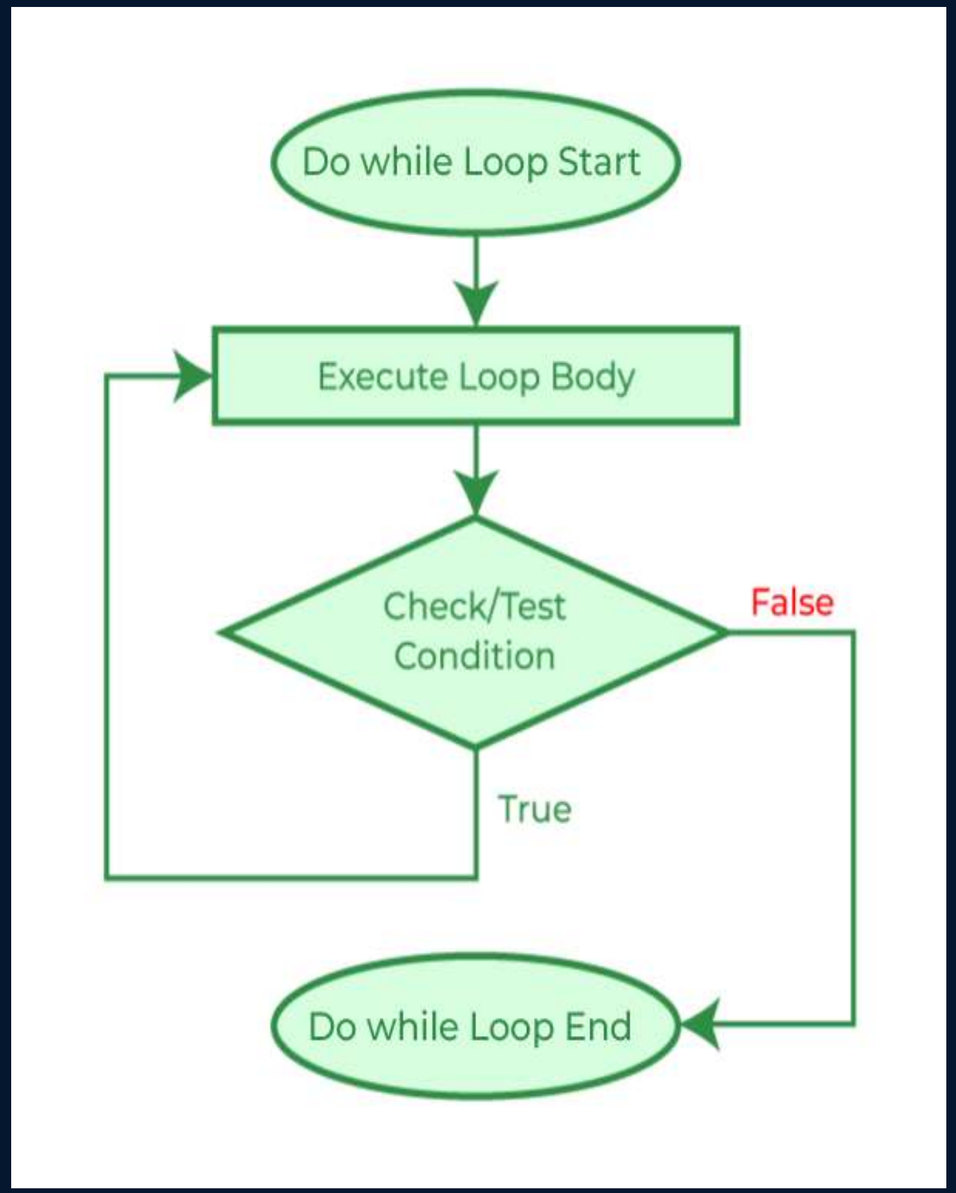


Fig:-Flow diagram for do while loop.

@Er_BHISMARAJ

iv) Recursive logic

- Recursive logic involves a function calling itself to solve a smaller instance of the same problem, progressively breaking it down until reaching a base case that can be directly solved. For example, consider the calculation of the factorial of a number (n!), which is the product of all positive integers up to n. Using recursion, the factorial function can be defined as:

$$\text{factorial}(n) = n \times \text{factorial}(n-1)$$

So, to find factorial of 5, the function repeatedly calls itself with decrements of 1 (5 * 4 * 3 * 2 * 1) until it reaches the base case.

Compiled by ER BJ

@Er_BHISMARAJ

THANK YOU..

Compiled by Er. BRJ

@Er_BHISMARAJ